**FILMORA**

**DEPLOYMENT, HOSTING & MOVIE STORAGE ARCHITECTURE**

**Project Name:** FILMORA
**Document Number:** FILMORA-DEP-007
**Version:** 1.0
**Status:** Proposed — Architecture Selected
**Video Hosting Provider:** Cloudflare Stream
**Backend:** Node.js + Express.js
**Database:** PostgreSQL
**Frontend:** HTML5, CSS3 and JavaScript
**Streaming Protocol:** HLS
**Initial Audience:** Unknown
**Deployment Strategy:** Scalable Cloud Infrastructure

---

**TABLE OF CONTENTS**


1. Introduction
2. Infrastructure Objectives
3. Hosting Architecture
4. Movie Storage Architecture
5. Cloudflare Stream Integration
6. Movie Upload Workflow
7. Video Processing
8. Video Streaming Workflow
9. Database Integration
10. Movie Metadata Storage
11. Secure Video Access
12. Video Player Integration
13. Administrator Upload Interface
14. Storage and Delivery Costs
15. Scalability Strategy
16. Monitoring and Analytics
17. Backup and Recovery

18. Deployment Requirements
19. Infrastructure Security
20. Final Architecture Decisions

---

**1. INTRODUCTION**

FILMORA will use Cloudflare Stream as its primary video hosting and delivery solution.

The platform will not store full-length movie files on its application server or directly inside PostgreSQL.

Instead, Cloudflare Stream will handle video storage, encoding and streaming delivery.

PostgreSQL will store movie information and the corresponding Cloudflare video identifiers.

**2. INFRASTRUCTURE OBJECTIVES**

The infrastructure must:

1. Store full-length movies securely.
2. Support adaptive video streaming.
3. Support multiple concurrent viewers.
4. Avoid unnecessary server maintenance.
5. Protect video access.
6. Support administrator uploads.
7. Track video processing status.
8. Support increasing audience sizes.
9. Monitor hosting expenses.
10. Maintain reliable video availability.

**3. HOSTING ARCHITECTURE**

The infrastructure will contain four principal components.

| Component | Provider / Technology | Responsibility |
|---|---|---|
| Frontend hosting | To be selected | Website delivery |
| Backend hosting | Node.js hosting provider | API and application logic |
| Database hosting | PostgreSQL hosting provider | Structured information |

| Component | Provider / Technology | Responsibility |
|---|---|---|
| Video hosting | Cloudflare Stream | Movie storage and streaming |

**4. MOVIE STORAGE ARCHITECTURE**

Cloudflare Stream will store the actual movie files.

PostgreSQL will store the associated movie records.

The FILMORA application server will not be responsible for transferring every movie file to viewers.

This separation reduces application server workload.

**4.1 Storage responsibilities**

**Cloudflare Stream stores:**

- Original uploaded video.
- Encoded streaming versions.
- Streaming manifests.
- Video processing information.

**PostgreSQL stores:**

- Movie title.
- Movie description.
- Release year.
- Duration.
- Genre.
- Poster reference.
- Backdrop reference.
- Cloudflare video UID.
- Processing status.
- Publication status.
- Content licensing information.

**5. CLOUDFLARE STREAM INTEGRATION**

FILMORA will integrate with the Cloudflare Stream API.

The backend will communicate with Cloudflare using protected API credentials.

The API credentials must remain on the server.

The frontend must never receive the Cloudflare API secret.

**5.1 Main integration responsibilities**

- Create video upload requests.
- Receive Cloudflare video identifiers.
- Track video processing.
- Receive processing notifications.
- Generate authorized playback access.
- Retrieve video information.
- Delete or manage videos when authorized.

**6. MOVIE UPLOAD WORKFLOW**

The administrator upload process will follow these steps:

1. Administrator logs into FILMORA.
2. Administrator opens the movie upload page.
3. Administrator enters movie information.
4. Administrator selects the movie file.
5. Backend verifies administrator permissions.
6. Backend requests a one-time upload URL from Cloudflare.
7. Frontend uploads the movie directly to Cloudflare.
8. Cloudflare processes the uploaded movie.
9. Cloudflare sends a processing notification to FILMORA.
10. Backend verifies the notification.
11. PostgreSQL receives the updated video status.
12. Administrator reviews the movie.
13. Administrator publishes the movie.

The movie must not become publicly available before processing succeeds and content authorization is confirmed.

**7. VIDEO PROCESSING**

Cloudflare Stream will handle video encoding.

FILMORA will not initially require a separate video encoding server.

Supported playback qualities will depend on the original video and Cloudflare's processing capabilities.

Proposed playback resolutions:

- 360p.
- 480p.
- 720p.
- 1080p.

Cloudflare provides adaptive bitrate streaming.

**8. VIDEO STREAMING WORKFLOW**

When a registered user wants to watch a movie:

1. User opens the movie details page.
2. User clicks Watch Now.
3. Frontend requests playback authorization.
4. Backend verifies the user's authenticated session.
5. Backend verifies movie publication status.
6. Backend checks content availability and licensing restrictions.
7. Backend generates temporary signed playback access.
8. Frontend receives the authorized playback information.
9. Video player requests the movie from Cloudflare.
10. User watches the movie.

**9. DATABASE INTEGRATION**

PostgreSQL will maintain the relationship between movie records and Cloudflare video assets.

Each movie will have a unique internal database identifier and a Cloudflare video UID.

The Cloudflare UID will be used to identify the corresponding video.

**10. MOVIE METADATA STORAGE**

The movies table will include the following proposed fields:

| Field | Data Type | Purpose |
|---|---|---|

| Field | Data Type | Purpose |
|---|---|---|
| id | UUID | Internal movie ID |
| title | VARCHAR | Movie title |
| description | TEXT | Movie description |
| release_date | DATE | Release date |
| duration | INTEGER | Duration in seconds |
| cloudflare_video_uid | VARCHAR | Cloudflare video identifier |
| processing_status | VARCHAR | Processing state |
| publication_status | VARCHAR | Publication state |
| requires_signed_urls | BOOLEAN | Access protection |
| created_at | TIMESTAMP | Creation date |
| updated_at | TIMESTAMP | Last update |

The Cloudflare video UID must be unique when present.

**11. SECURE VIDEO ACCESS**

Movie access must be restricted to authorized users.

FILMORA will use Cloudflare signed URLs or signed playback tokens.

Requirements:

- Authenticated user verification.
- Movie availability verification.
- Short-lived playback authorization.
- Restricted video access.
- Secure API credentials.
- Appropriate session expiration.

Public movie playback must be disabled for protected content.

**12. VIDEO PLAYER INTEGRATION**

The frontend will use:

- HTML5 Video.

- HLS.js where necessary.
- Cloudflare Stream HLS playback.
- Responsive video controls.

The player must support:

- Play and pause.
- Fullscreen.
- Volume.
- Playback progress.
- Seeking.
- Subtitles where available.
- Audio tracks where available.
- Adaptive quality.

**13. ADMINISTRATOR UPLOAD INTERFACE**

The administration dashboard will contain an upload form.

Required information:

- Movie title.
- Description.
- Release date.
- Duration.
- Genre.
- Poster.
- Backdrop.
- Movie file.
- Content rights information.

The interface will display:

- Upload progress.
- Processing progress where available.
- Upload status.
- Processing errors.

- Publication status.

**14. STORAGE AND DELIVERY COSTS**

Cloudflare Stream uses two main billing dimensions.

**A. Stored video minutes**

The published pricing is $5 per 1,000 minutes of stored video per month.

**B. Delivered video minutes**

The published pricing is $1 per 1,000 minutes delivered.

Uploading and encoding are included without separate ingress or encoding charges.

Illustrative examples:

| Usage | Estimated Cost |
|---|---|
| 1,000 minutes stored | $5/month |
| 5,000 minutes stored | $25/month |
| 10,000 minutes stored | $50/month |
| 10,000 minutes delivered | $10 |
| 100,000 minutes delivered | $100 |

These are usage calculations, not a complete monthly hosting budget. Actual bills depend on storage capacity and total playback delivery.

**15. SCALABILITY STRATEGY**

Because the initial audience is unknown, FILMORA will use a scalable architecture.

Initial strategy:

- Start with managed video hosting.
- Monitor storage consumption.
- Monitor delivered video minutes.
- Monitor concurrent playback.
- Monitor backend response times.
- Track database performance.
- Increase infrastructure capacity when necessary.

Infrastructure decisions will be based on actual usage rather than guessed audience numbers.

**16. MONITORING AND ANALYTICS**

The platform should monitor:

- Number of uploaded movies.
- Video processing failures.
- Total stored video minutes.
- Total delivered video minutes.
- Playback errors.
- Video startup times.
- Viewing activity.
- Upload failures.
- Monthly infrastructure expenses.

**17. BACKUP AND RECOVERY**

FILMORA will maintain:

- PostgreSQL backups.
- Database restoration procedures.
- Media metadata backups.
- Cloudflare video inventory records.
- Secure API configuration backups.

Cloudflare Stream will be the primary video storage provider, but important original movie files and legally required archival copies should also be maintained independently where feasible.

**18. DEPLOYMENT REQUIREMENTS**

The production environment must provide:

- Secure frontend hosting.
- Backend hosting.
- PostgreSQL hosting.
- Cloudflare Stream integration.
- HTTPS.
- Environment variables.
- Monitoring.

- Automated backups.
- Error reporting.

**19. INFRASTRUCTURE SECURITY**

Security requirements:

- Protect API tokens.
- Restrict administrator uploads.
- Use signed playback access.
- Validate uploaded files.
- Verify Cloudflare webhook signatures.
- Restrict database permissions.
- Use HTTPS.
- Monitor suspicious activity.

**20. FINAL ARCHITECTURE DECISIONS**

The following decisions are approved for the initial FILMORA architecture:

| **Component** | **Selected Technology** |
|---|---|
| Frontend | HTML, CSS, JavaScript |
| Backend | Node.js + Express.js |
| Database | PostgreSQL |
| ORM | Prisma |
| Video Hosting | Cloudflare Stream |
| Video Processing | Cloudflare Stream |
| Streaming | HLS |
| Playback Authorization | Signed URLs / Tokens |
| API Architecture | REST API |
| Initial Audience | Unknown |
| Architecture | Modular Monolith |

---

**FINAL INFRASTRUCTURE PRINCIPLE**

FILMORA will use Cloudflare Stream to store, encode and deliver full-length movies.

The application backend will manage authentication, authorization, movie information and streaming access, while PostgreSQL will maintain structured platform data.

This architecture minimizes video infrastructure complexity and allows the platform to scale according to actual audience demand.

**DOCUMENT STATUS: PROPOSED — ARCHITECTURE SELECTED**

**FILMORA — YOUR WORLD OF MOVIES.**